



§ 5. 数组

5.7. C++处理字符串的方法 - 字符串类与字符串变量

5.7.1. 用一维字符数组来表示字符串变量的不足

★ 字符串的长度受限于数组定义时的大小

=> 数组定义过大导致浪费

=> 数组定义过小导致不够

=> 数组定义的大小不能动态变化

★ 赋值、复制、连接等必须用专用函数(strcpy/strcat/...)

5.7.2. 字符串类(string类)的引入

string类是C++引入的字符串处理的新方法，通过定义类及运算符重载的方式实现

★ string类不是C++的原生数据类型

● 判断是否原生的标准：无任何头文件时定义某种数据类型的变量

```
demo.cpp
1 int main()
2 {
3     int x = 0;
4     return 0;
5 }
6
```

输出

生成开始于 15:00...

1> 已启动生成: 项目: demo-cpp, 配置: Debug x64

1> demo.cpp

1> demo-cpp.vcxproj -> B:\Workspace\VS2026-Demo\Debug\demo-cpp.exe

生成: 1 成功, 0 失败, 0 最新, 0 已跳过

生成于 15:00 完成, 耗时 00.449 秒

原生

非原生

```
demo.cpp
1 int main()
2 {
3     string s = "";
4     return 0;
5 }
6
```

输出

生成开始于 15:23...

1> 已启动生成: 项目: demo-cpp, 配置: Debug x64

1> demo.cpp

1> B:\Workspace\VS2026-Demo\demo-cpp\demo.cpp(3,5): error C2065: "string": 未声明的标识符

1> B:\Workspace\VS2026-Demo\demo-cpp\demo.cpp(3,12): error C2146: 语法错误: 缺少"{" (在标识符"s"的前面)

1> B:\Workspace\VS2026-Demo\demo-cpp\demo.cpp(3,12): error C2065: "s": 未声明的标识符

生成: 0 成功, 1 失败, 0 最新, 0 已跳过

生成于 15:23 完成, 耗时 01.066 秒



§ 5. 数组

5.7. C++处理字符串的方法 - 字符串类与字符串变量

5.7.3. string类简单变量的定义和使用（与C方式对比）

项目	字符串变量	字符数组
适用	C++	C、C++
头文件	#include <string>	#include <string.h> #include <cstring>
定义	string 变量名 string s1;	char 变量名 char s1[10];
定义时赋初值	string s1="hello"; 长度无限制	char s1[10]="hello"; 长度不超过9
赋值	string s1; s1="hello"; 长度无限制 string s1="hello", s2; s2=s1;	char s1[10]; strcpy(s1, "hello"); 长度不超过9 char s1[10]="hello", s2[10]; strcpy(s2, s1);
单字符操作	string s1="hello"; s1[2]='p';	char s1[10]="hello"; s1[2]='p';
输入	cin	cin、scanf
输出	cout	cout、printf
字符串复制	string s1, s2; ... s1=s2; 不必考虑溢出	char s1[10], s2[10]; ... strcpy(s1, s2); 防止溢出
字符串连接	string s1, s2; ... s1=s1+s2; 不必考虑溢出	char s1[20], s2[10]; ... strcat(s1, s2); 防止溢出
	注: s1+s2 ≠ strcat ; s1=s1+s2/s1+=s2 ⇔ strcat	
字符串比较	用比较运算符 s1==s2; s1>s2; s1>=s2;	用函数 if (!strcmp(s1, s2)) if (strcmp(s1, s2)>0) if (strcmp(s1, s2)>=0)



§ 5. 数组

5.7. C++处理字符串的方法 - 字符串类与字符串变量

5.7.3. string类简单变量的定义和使用（与C方式对比）

例：键盘输入三个字符串，按字典序从小到大输出（上：C方式字符数组/下：C++的string类）

```
int main()
{
    char str[3][80];
    for (int i=0; i<3; i++)
        cin >> str[i]; //二维数组带一个下标，长度不超过79
    if (strcmp(str[0],str[1])<=0 && strcmp(str[1],str[2])<=0) //1 < 2 < 3
        cout << str[0] << ' ' << str[1] << ' ' << str[2] << endl;
    else if //剩下五种情况
        ...
}
```

C方式一维字符数组表示字符串

两者的总体思路没有区别，
仅具体语句有区别

```
int main()
{
    string str[3];
    for (int i=0; i<3; i++)
        cin >> str[i]; //string一维数组，长度任意
    if ( str[0] <= str[1] && str[1] <= str[2] ) //1 < 2 < 3，直接比较
        cout << str[0] << ' ' << str[1] << ' ' << str[2] << endl;
    else if //剩下五种情况
        ...
}
```

C++方式string表示字符串



§ 5. 数组

5.7. C++处理字符串的方法 - 字符串类与字符串变量

5.7.3. string类简单变量的定义和使用（与C方式对比）

//观察string的所占空间及存放的字符串长度的关系

```
#include <iostream>
using namespace std;
int main()
{
    string s1 = "abc";
    string s2 = "abcdefghijklmnopqrstuvwxyz01234567890123456789";
    cout << sizeof(s1) << ' ' << s1.length() << endl;
    cout << sizeof(s2) << ' ' << s2.length() << endl;
    for (int i = 0; i < 10000; i++)
        s1 += s2;
    cout << sizeof(s1) << ' ' << s1.length() << endl;
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
40 3
40 46
40 460003
```

现象：大小40字节的变量存储了超过40字节的内容

2000
2039

S1

大于460003字节
的连续空间

- ★ string变量不是原生的基本数据类型，是C++封装的一种复杂数据类型
- ★ string变量存放的字符串的长度会自动调整，不是固定大小
- ★ string变量的sizeof大小与存放串的长度无关，涉及到一种新的内存存储和分配方式(动态内存的申请与释放-后续课程)
- ★ string变量能用+、=、>等运算符来实现连接、赋值、比较等运算，具体的实现原理及实现方法待后续内容(运算符的重载-后续课程)学习完成后再进一步了解
- ★ 在本节中仅需要了解与一维字符数组表示字符串在表示及使用方法的差异即可
- ★ 除特定题目外，string类本学期都不准使用



§ 5. 数组

5.7. C++处理字符串的方法 – 字符串类与字符串变量

5.7.4. string类数组的定义和使用（与C方式对比）

★ 形式

```
string 数组名[正整型常量表达式]; //一维  
string 数组名[正整型常量表达式1][正整型常量表达式2]; //二维  
string name[5];  
string course[3][4];
```

★ 含义

数组有若干元素，每个元素是一个任意长度的string类变量

★ 定义时赋初值

```
string name[5]={"Zhang", "Li", "Wang", "Lin", "Zhao"};  
string course[3][4]={ {"C语言", "C++", "Pyhton", "Java"},  
                        {"高数", "数分", "高代", "线代"},  
                        {"足球", "篮球", "排球", "游泳"}  
};
```

- 对一维数组，字符串的总数不超过数组大小
- 对二维数组，内括号内字符串个数不超过列，内括号总数不超过行
- 字符串的长度不受限制



§ 5. 数组

5.7. C++处理字符串的方法 - 字符串类与字符串变量

5.7.4. string类数组的定义和使用（与C方式对比）

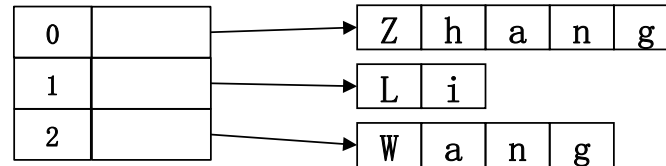
★ 与二维字符数组的内存表示比较(定义时赋初值)

```
char str[3][30]={"Zhang", "Li", "Wang"};
```

Z	h	a	n	g	\0		\0
L	i	\0	\0	\0	\0		\0
W	a	n	g	\0	\0		\0

str占用连续的90个字节

```
string name[3]={"Zhang", "Li", "Wang"};
```



name数组占用连续的120字节，name[0]、name[1]、name[2]中的串分别占用连续的5/2/4个字节，但整体上不保证连续

★ string类一维数组与二维字符数组的比较(输入/输出)

```
char str[3][30];
string name[3];
int i;
for (i=0; i<3; i++) {
    cin >> str[i]; //二维数组带单下标，输入长度不超过29
    cin >> name[i]; //一维数组带下标，输入长度不限
    cout << str[i] << '-' << name[i] << endl;
}
```

```
char str[3][30];
string name[3];
strcpy(str[0], "Zhang");
strcpy(str[1], "Li");
strcpy(str[2], "Wang");
name[0] = "Zhang";
name[1] = "Li";
name[2] = "Wang";
```

专用函数strcpy
且串长不超过29

赋值运算符
长度不限

★ string类一维数组与二维字符数组的比较(通过strcpy/=进行赋值)